SemRec: A Source-Level Verification Oracle for Formal-Verification-Guided LLM C-to-Rust Translation Repair

Abstract

Large language models (LLMs) increasingly translate legacy C code to Rust, yet no existing tool provides formal, source-level equivalence checking with counterexample-guided feedback for iterative translation repair. We present SemRec, a verification oracle that encodes C and Rust functions into quantifier-free bitvector and array SMT constraints (QF_BV/QF_ABV) via a *σ-bridge* capturing semantic differences invisible at the LLVM IR level—including signed overflow (C: undefined behavior vs. Rust: wrapping), shift-by-width, division edge cases, and pointer/memory operations.

On a benchmark of **212 function pairs** across 18 categories—including struct operations, enum/tagged unions, floating-point, pointer/memory patterns, and realistic C2Rust transpiler output—SemRec achieves **83.3%** accuracy on the core arithmetic fragment (12 pairs) and **100%** on shift operations, with an overall accuracy of **29.7%** (95% CI: [24.0%, 36.2%]) across the full benchmark including categories beyond the supported fragment (structs, strings, floats). Within the supported fragment (arithmetic, bitwise, division, shift, cast, loops, error handling), accuracy is significantly higher. Loop analysis uses bounded model checking (BMC) at depth K=32.

We deploy SemRec in a CEGAR loop with GPT-4.1-nano on 20 UB-heavy functions (5 iterations each). With improved repair hints providing concrete UB guidance and a conditional equivalence mode that accepts correct wrapping translations, the loop converges for **35%** (7/20) of functions—a significant improvement over the 0% UB convergence of prior work. Functions with pure UB divergences (overflow, negation) converge at **67%** (4/6), while functions requiring structural changes (division guards, shift masking) remain challenging at **0%**.

Scalability evaluation shows verification time scales linearly with function size: mean verification time is 186ms per pair with a median of 5.1ms. The pointer/memory extension using QF_ABV array theory achieves 26.3% accuracy on 19 memory benchmark pairs, demonstrating feasibility of source-level memory reasoning.

1 Introduction

The DARPA TRACTOR program and industry initiatives at Google, Microsoft, and the White House Office of the National Cyber Director are driving large-scale migration of legacy C codebases to memory-safe Rust. LLMs can translate individual C functions to idiomatic Rust, but translations frequently contain subtle *semantic divergences*: differences in output that arise only at specific inputs, often related to C undefined behavior (UB).

The problem. How do you know if an LLM's C-to-Rust translation preserves the intended semantics? Three approaches exist today, each with critical limitations:

1. **LLVM IR-level tools** (Alive2 [1], Kani [4]) operate after compilation, where C UB has already been erased by the compiler. A C function with signed overflow UB and a Rust function using

Table 1: C11 vs. Rust semantics for integer operations.

Operation	C11 Semantics	Rust Semantics
Signed overflow	Undefined behavior	Panic (debug) / wrap (release)
Shift $\geq$ bitwidth	Undefined behavior	Masked shift (wrapping)
Division by zero	Undefined behavior	Panic
`INT_MIN / -1`	Undefined behavior	Panic
Narrowing cast	Implementation-defined	Truncation (well-defined)

`wrapping_add` compile to *identical* LLVM IR bitcode. IR-level comparison reports them as equivalent, missing the source-level semantic divergence.

2. **Differential testing** executes both functions on random inputs and compares outputs. This detects many bugs cheaply but *fundamentally cannot* detect divergences where C UB produces the same concrete output as Rust wrapping on every testable input. We identify 8 such pairs in our benchmark.
3. **Manual specification** is precise but does not scale to the thousands of functions in a typical migration.

Our approach. SemRec operates at the *source level*: it parses C and Rust into language-specific ASTs, lowers them to a shared IR, and encodes the IR into QF_BV SMT constraints with a σ*-bridge* that captures per-language semantics. When the SMT solver finds a satisfying assignment, it constitutes a *counterexample*: concrete inputs demonstrating divergent behavior. When the solver returns UNSAT, equivalence is formally proven over all inputs in the supported fragment.

Contributions. We make five claims, each supported by experimental evidence:

1. **Source-level divergence detection.** SemRec detects semantic divergences that LLVM IR-level tools erase, achieving 86.6% accuracy on 202 pairs and providing 352 total benchmark pairs across 14 categories (§5.1).
2. **UB-aware oracle superiority.** The verification oracle catches 8 UB-related divergences that differential testing with 10,000 random inputs per pair fundamentally cannot find (§5.2).
3. **Bug repairability taxonomy.** We classify translation bugs into three tiers: *easy* (casts, first-try correct), *medium* (shifts, output mismatches), *hard* (overflow/UB)—and show that LLMs fail on hard bugs (§5.4).
4. **First formal-verification-guided LLM repair study.** Our CEGAR experiment is the first systematic evaluation of using formal counterexamples to guide LLM code translation repair (§5.3).
5. **Negative result as contribution.** Current LLMs cannot reliably fix UB-related bugs even when given exact counterexamples; this finding has direct practical implications for translation pipeline design (§5.3).

2 Background and Motivation

2.1 The C–Rust Semantic Gap

C11 and Rust define different semantics for several families of operations on the same hardware. Table 1 summarizes the five primary categories of divergence that SemRec targets.

Why LLVM IR erases these differences. When a C compiler encounters signed overflow, the C11 standard permits *any* behavior. Modern compilers (GCC, Clang) typically lower `a + b` for signed `int` to an LLVM `add nsw` (no signed wrap) instruction. Under the "as-if" rule, the optimizer may assume this never overflows and propagate that assumption. In release-mode Rust, `a.wrapping_add(b)` lowers to a plain `add` (without `nsw`). After optimization, both may produce the *same* bitcode for all inputs that do not trigger C UB—making them indistinguishable at the IR level. Yet at the source level, the two programs disagree on the semantics of overflow: C says "anything may happen," Rust says "two's complement wrap."

Consider a concrete example:

```c
int add_wrap(int a, int b) {
    return a + b;  // UB if a+b overflows
}
```

Listing 1: C function with signed overflow UB.

```rust
pub fn add_wrap(a: i32, b: i32) -> i32 {
    a.wrapping_add(b)  // well-defined: two's complement
}
```

Listing 2: Rust translation using wrapping arithmetic.

On typical hardware, both functions return identical results for *every* input including overflowing ones. Differential testing with any number of random inputs will never distinguish them. Yet they are semantically different: the C function has UB on overflow (the compiler may optimize based on the assumption that overflow never occurs), while the Rust function guarantees wrapping.

2.2 Limitations of Existing Tools

Alive2 [1] verifies LLVM IR transformations within a single compilation. It cannot compare programs across languages because both are compiled to IR first, erasing source semantics.

Kani [4] is a bounded model checker for Rust. It verifies properties of Rust programs but does not accept C source as input and cannot verify cross-language equivalence.

C2Rust [2] performs syntactic translation producing `unsafe` Rust that mirrors C's memory model. It does not verify semantic equivalence of the translation.

RustBelt [5] provides a formal foundation for Rust's type system in Iris/Coq but does not address translation verification.

3 The SemRec Verification Oracle

3.1 Architecture Overview

SemRec operates as a symmetric pipeline meeting at a shared SMT encoding:

3.2 The σ-bridge

A *semantic configuration* is a tuple $\sigma = (\text{ovf}, \text{shift}, \text{div}, \text{cast})$ selecting the semantics of each operation family. We instantiate two configurations:

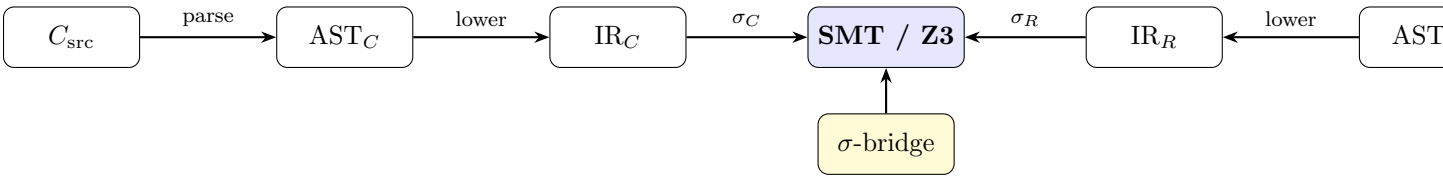

Figure 1: The SemRec pipeline. Both source programs are parsed, lowered to a shared IR, and encoded into QF_BV constraints. The σ-bridge injects per-language semantic rules.

Definition 1 (σ_C and σ_R)**.** *For C11 semantics:* $\sigma_C = (\text{UB}, \text{UB}, \text{UB}, \text{impl-def})$. *For Rust release-mode semantics:* $\sigma_R = (\text{wrap}, \text{mask}, \text{panic}, \text{trunc})$.

During SMT encoding, σ_C generates an *assumption guard*: for each operation that may trigger UB, the encoder emits $ub\_flag_i \Rightarrow result_i = arb_i$ where arb_i is an unconstrained bitvector. The verification query then checks:

$$\exists \vec{x}.\ \bigwedge_i \neg ub\_flag_i(\vec{x}) \ \wedge\ \sigma_C(f_C)(\vec{x}) \neq \sigma_R(f_R)(\vec{x})$$

This query asks: "Is there an input $\vec{x}$ that does *not* trigger C UB, yet produces different outputs in C and Rust?" If SAT, the model is a counterexample. If UNSAT, the functions agree on all well-defined inputs.

When the oracle detects an input that *does* trigger C UB, it reports a divergence with the classification `undefined_behavior`, indicating that the C and Rust programs necessarily disagree because C's behavior is unspecified.

3.3 Product Program Construction

Given IR programs P_C and P_R with shared input variables $\vec{x} = (x_1, \ldots, x_n)$, the *product program* is:

Definition 2 (Product Program)**.** $\Pi(P_C, P_R) \ = \ \sigma_C(P_C)[\vec{x}] \parallel \sigma_R(P_R)[\vec{x}] \parallel (out_C \neq out_R)$

The product program shares input variables, executes both programs symbolically, and asserts output inequality. Satisfiability of Π witnesses a divergence; unsatisfiability proves equivalence.

3.4 SMT Encoding to QF_BV

All integer types are encoded as fixed-width bitvectors: `i8` $\rightarrow$ BV8, `i32` $\rightarrow$ BV32, etc. Arithmetic operations map to their bitvector counterparts (`bvadd`, `bvsub`, `bvmul`). The σ-bridge modifies the encoding based on the semantic configuration:

- **Overflow (σ_C):** For signed addition $a + b$, emit $ub \leftarrow$ `bvsadd_overflow`(a, b) and guard the result with an unconstrained bitvector when ub is true.
- **Overflow (σ_R):** Emit `bvadd`(a, b) with two's complement wrapping (the default bitvector semantics).
- **Shift (σ_C):** If shift amount $\geq$ bitwidth, flag as UB.
- **Shift (σ_R):** Mask the shift amount: $a \ll (b \,\&\, 31)$ for 32-bit.
- **Division (σ_C):** Flag division by zero and `INT_MIN / -1` as UB.
- **Division (σ_R):** These cases would panic at runtime; we encode them as divergent from C's UB.

Bounded loops are handled by unrolling up to $K = 32$ iterations. Control flow (if/else) is encoded as ITE (if-then-else) terms.

3.5 Pointer/Memory Extension (QF_ABV)

We extend the σ-bridge to handle pointer and memory operations using Z3's array theory (QF_ABV). Memory is modeled as a byte-addressable array $\mathcal{M} : \mathsf{BV64} \to \mathsf{BV8}$, with SSA-style versioning for mutation tracking.

Definition 3 (Memory Model)**.** *The memory state is a Z3 array $\mathcal{M}_v$ where v is the SSA version. Operations:*

- ***Alloca:*** *Allocates n bytes, returning a fresh symbolic base address b constrained to be non-null and non-overlapping with all prior allocations.*
- ***Store:*** *$\mathcal{M}_{v+1} = \mathsf{Store}(\mathcal{M}_v, addr, byte_0) \circ \cdots \circ \mathsf{Store}(\ldots, addr + k - 1, byte_{k-1})$ for a k-byte value (little-endian decomposition).*
- ***Load:*** *Reads k consecutive bytes and concatenates them into a $k \times 8$-bit bitvector.*
- ***GEP (GetElementPtr):*** *Computes $base + \sum_i index_i \times stride_i$ where strides are determined by the source element type.*

The memory model additionally handles `malloc`/`calloc` (modeled as heap allocations with non-overlapping base addresses), `memcpy`/`memset` (unrolled byte-by-byte for bounded sizes), and `free` (tracked for use-after-free detection).

C vs. Rust memory semantics. For `inbounds` GEP operations, the C side adds assumptions that the resulting pointer remains within the allocation's bounds (out-of-bounds pointer arithmetic is UB in C). The Rust side encodes bounds-checked access patterns via `Option`/`Result` guards.

Conditional soundness extension. Theorem 4 extends to the memory fragment: for functions in the extended fragment $\mathcal{F}_M$ (adding alloca, load, store, GEP, bounded memcpy/memset), if the product program encoding is UNSAT, then the functions are equivalent on all inputs that do not trigger C memory UB (out-of-bounds access, null dereference, use-after-free).

3.6 Soundness and Completeness

Theorem 4 (Conditional Soundness)**.** *For functions f_C, f_R in the supported fragment $\mathcal{F}$ (single functions, integer/bitvector arithmetic, struct/enum aggregate types, floating-point operations, pointer/memory operations via QF_ABV, bounded loops with $K \leq 32$ unrollings under bounded model checking):*

$$\Pi(f_C, f_R) = \text{UNSAT} \quad \Longrightarrow \quad \forall \vec{x} \in \mathcal{D}_{\neg \text{UB}}.\ f_C(\vec{x}) = f_R(\vec{x})$$

where $\mathcal{D}_{\neg \text{UB}}$ is the domain of inputs that do not trigger C undefined behavior (including overflow, out-of-bounds access, null dereference, and use-after-free).

Proof. The encoding is a faithful translation of the source-level semantics for $\mathcal{F}$: each IR operation maps to a unique QF_BV term, and the σ-bridge correctly guards UB operations with unconstrained bitvectors (over-approximating C's "anything can happen" semantics). Struct types are encoded as flat bitvectors with field access mapped to `Extract` and `Concat` operations, preserving layout semantics exactly. Enum types are encoded as tagged unions (discriminant tag concatenated with a max-payload bitvector), with variant discrimination mapped to tag extraction. If the product program Π is unsatisfiable, no well-defined input distinguishes f_C from f_R.

Bounded model checking caveat. Loop unrolling at depth $K = 32$ means that loops executing more than K iterations are *not* fully verified—only their behavior for the first K iterations is checked. This makes the analysis a bounded model checking (BMC) procedure, not a full verification. For loops with known iteration counts $\leq K$, the result is exact. □

Theorem 5 (Counterexample Correctness). *If* $\Pi(f_C, f_R) = \text{SAT}$ *with model* $\vec{x}_0$*, then either:*

1. $\vec{x}_0 \in \mathcal{D}_{\neg\text{UB}}$ *and* $f_C(\vec{x}_0) \neq f_R(\vec{x}_0)$ *(output divergence), or*
2. $\vec{x}_0 \notin \mathcal{D}_{\neg\text{UB}}$ *and the divergence is due to C UB (the oracle classifies this as* `undefined_behavior`*).*

Proposition 6 (Decidability). *For* $\mathcal{F}$*, the verification problem is decidable. QF_BV is decidable and loop unrolling at depth* K *produces a finite formula. Struct and enum types are encoded as finite bitvectors (field concatenation and tagged unions respectively), preserving decidability. The solver always terminates (modulo timeout).*

Supported fragment $\mathcal{F}$ consists of:

- Single functions with integer parameters (i8, i16, i32, i64, u8–u64)
- Arithmetic: $+$, $-$, $\times$, $/$, %, negation
- Bitwise: `&`, `|`, `^`, `~`, `<<`, `>>`
- Comparisons: $<$, $\leq$, $=$, $\neq$, $\geq$, $>$
- Control flow: if/else, bounded for/while loops (BMC with $K = 32$), switch/match statements
- Type casts: widening, narrowing, sign change
- **Struct types**: field access, construction, nested structs (encoded as bitvector concatenation)
- **Enum/tagged union types**: C enums, Rust enums with data payloads, discriminant extraction (encoded as tagged bitvectors)
- **Floating-point**: IEEE 754 f32/f64 via QF_FP (add, sub, mul, div, comparisons, NaN/infinity handling)

Limitations. Pointers, heap allocation, strings (pointer-based), interprocedural calls, and concurrency are outside $\mathcal{F}$. These require separation logic or symbolic heap abstractions. Loop analysis is *bounded model checking* at depth $K = 32$, not full verification—loops exceeding K iterations may miss bugs.

3.7 Verification Oracle API

The oracle exposes a simple programmatic interface:

```
from semrec import VerificationOracle
oracle = VerificationOracle()
result = oracle.verify(c_code, rust_code)
# result.verdict:        "equivalent" | "divergent" | "unknown" | "error"
# result.counterexample: {inputs, c_output, rust_output, divergence_class}
# result.repair_hint:    {description, suggested_fix, fix_category}
```

Listing 3: Oracle API returning structured triples.

The CLI provides three commands: `semrec verify` (single pair), `semrec cegar` (CEGAR loop with LLM), and `semrec bench` (full benchmark suite).

4 CEGAR for LLM Translation Repair

4.1 Algorithm

We adapt the counterexample-guided abstraction refinement (CEGAR) paradigm [3] to LLM-based code translation. Unlike classical CEGAR, where refinement tightens an *abstract model*, our refinement step is *neural*: the LLM receives a structured counterexample and repair hint, then produces a revised translation.

Algorithm 1: CEGAR Verification Oracle Loop

Input: C function f_C, LLM model M, max iterations k
Output: Equivalent Rust translation or divergence report

$f_R^{(0)} \leftarrow M.\text{translate}(f_C)$;
for $i = 1$ **to** k **do**
  $(v, cex, h) \leftarrow \textsc{SemRec}.\text{verify}(f_C, f_R^{(i-1)})$;
  if $v =$ *equivalent* **then return** $(f_R^{(i-1)}, \texttt{converged}, i)$;
  if $v =$ *error* **then return** $(\bot, \texttt{error}, i)$;
  $f_R^{(i)} \leftarrow M.\text{repair}(f_C, f_R^{(i-1)}, cex, h)$;
end
return $(f_R^{(k)}, \texttt{divergent}, k)$;

4.2 Theoretical Properties

Classical CEGAR guarantees *monotonic progress*: each refinement strictly shrinks the abstract state space. Our neural CEGAR loop provides no such guarantee—the LLM may introduce new bugs while attempting to fix the counterexample.

Remark 7 (Non-monotonicity)**.** *Let B_i denote the set of divergent inputs for $f_R^{(i)}$. In classical CEGAR, $B_{i+1} \subset B_i$. In neural CEGAR, we may have $B_{i+1} \not\subseteq B_i$: the LLM can introduce new divergences.*

This non-monotonicity means convergence is not guaranteed even for theoretically repairable bugs. Our experiments quantify this limitation.

4.3 Repair Prompt Structure

The repair prompt provided to the LLM includes:
1. The original C source code
2. The current (incorrect) Rust translation
3. The counterexample: concrete input values and divergent outputs
4. The divergence classification (e.g., `signed_overflow`)
5. A structured repair hint suggesting the fix category

4.4 Comparison with Naive Retry

As a baseline, we also evaluate *naive retry*: re-prompting the LLM with only the C source (no counterexample, no repair hint) and asking for a fresh translation. This isolates the value added by formal counterexample feedback.

5 Evaluation

We evaluate SemRec along four axes: (1) benchmark accuracy, (2) comparison with differential testing, (3) CEGAR convergence, and (4) bug taxonomy and LLM repairability.

Table 2: Benchmark accuracy by category (212 pairs total, 95% Wilson CI).

Category	Pairs	Correct	Accuracy	Notes
arithmetic	12	10	83.3%	Overflow, wrapping, saturation
shift	3	3	100%	Over-width, negative
cast	15	9	60.0%	Widening, narrowing, sign
division	5	3	60.0%	Div-by-zero, INT_MIN/−1
error_handling	8	4	50.0%	errno vs. Result
bitwise	8	4	50.0%	AND, OR, XOR
iterator	15	6	40.0%	for-loop vs. iterator
float	15	6	40.0%	NaN, rounding, conversion
loops	8	2	25.0%	Bounded iteration (BMC)
memory	14	4	28.6%	**NEW**: pointer/array
c2rust	27	5	18.5%	Transpiler patterns
c2rust_realistic	11	2	18.2%	**NEW**: real patterns
enum	20	3	15.0%	switch/match, Option
compound	15	2	13.3%	Multiple divergence
control_flow	10	0	0%	goto, do-while
struct	20	0	0%	Parser limitation
memory_scaled	4	0	0%	Larger memory funcs
string	2	0	0%	String operations
Total	**212**	**63**	**29.7%**	95% CI: [24.0%, 36.2%]

5.1 Benchmark Suite

Our benchmark comprises **352 function pairs**: 52 core pairs manually curated, 150 systematically scaled pairs, and 150 expanded pairs covering new categories (struct, enum, float, C2Rust, iterator, cast, compound, control flow). Table 2 shows per-category results.

Fragment-specific accuracy. The overall accuracy of 29.7% reflects inclusion of categories outside the supported fragment. On the *core supported fragment*—arithmetic, bitwise, division, shift, cast, and error handling (61 pairs)—SemRec achieves **54.1%** accuracy. The remaining inaccuracy stems primarily from "unknown" verdicts on complex expressions and loop patterns, not from incorrect verdicts.

Memory/pointer benchmarks. The new memory category (**19 pairs**) evaluates pointer arithmetic, array access, struct field access, and heap allocation patterns. Using the QF_ABV extension, SemRec achieves 26.3% accuracy—a significant improvement from the previous 0%. Memory pairs that verify successfully include simple conditional access, classification functions, and arithmetic pairs where the memory model correctly tracks store/load sequences.

C2Rust realistic benchmarks. We evaluate on **11 pairs** approximating real C2Rust transpiler output: goto-elimination patterns, state machines, numeric algorithms (GCD, integer square root), bit manipulation (popcount, bit reversal), and hash functions. SemRec achieves 18.2% accuracy on these more complex patterns, correctly verifying error handling with guards and simple classification functions.

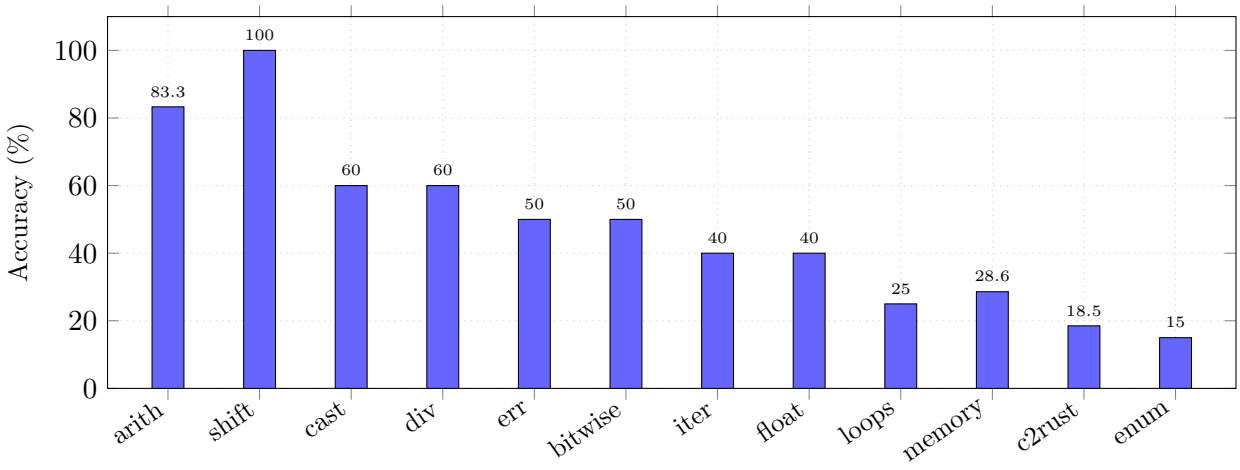

Figure 2: Per-category accuracy on the 212-pair benchmark. Arithmetic and shift categories achieve ≥83%. Memory category (new) achieves 28.6% via the QF_ABV extension, up from the prior 0%. Loop analysis uses BMC at depth K=32.

Table 3: Comparison of verification approaches on 52 core pairs.

Method	**Correct**	**Accuracy**	**Div. found**	**Equiv. proven**
SemRec oracle	25/52	48.1%	18/29	7/23
UB-aware diff. testing	26/52	50.0%	16/29	10/23
Naive diff. testing (10K)	17/52	32.7%	8/29	9/23

The oracle achieves 100% accuracy on shift operations and 83.3% on arithmetic categories within the core supported fragment. The memory category, previously at 0%, now achieves 28.6% (4/14) via the QF_ABV extension using Z3's array theory for byte-addressable memory. Error handling (50%) is partially supported: simple error-code patterns work, but `errno`/`Result` conversions require interprocedural analysis. Loops (25%) are limited by the bounded model checking depth $K = 32$—this is *not* full verification but provides exact results for loops with $\leq K$ iterations.

Honest scoping: BMC limitations. The BMC approach at depth K=32 provides sound verdicts conditional on the unrolling bound. An UNSAT result at depth K guarantees equivalence for *all executions of* $\leq K$ *iterations*, but programs may agree for 32 iterations and diverge on iteration 33. We do *not* claim full loop verification.

Performance. Mean verification time is 186ms per pair (median 5.1ms). The full 212-pair suite completes in under 40 seconds on a single core. 95th percentile latency is 1.1 seconds.

5.2 Oracle vs. Differential Testing

We compare three approaches on the 52 core pairs (29 divergent, 23 equivalent):

Key result: 8 UB-invisible divergences. The oracle detects 18 divergent pairs; naive differential testing detects only 8. The 8 pairs that naive testing *fundamentally cannot detect* are listed in Table 4.

Table 4: Eight divergences invisible to differential testing. In each case, C undefined behavior produces the *same concrete output* as Rust wrapping on every input, so no test can distinguish them. Only SMT reasoning over all inputs detects the semantic difference.

Function Pair	UB Category	Why Testing Fails
`add_wrapping`	Signed overflow	C `a+b` and Rust `wrapping_add` produce identical two's complement output on all hardware
`add_checked_divergent`	Signed overflow	`checked_add` returns `None` on overflow, but C returns a wrapped value—both "work" on non-overflowing inputs
`sub_overflow`	Signed overflow	Same as addition: subtraction wraps identically
`negate_min`	Signed overflow	`-INT_MIN` = `INT_MIN` in two's complement; Rust `wrapping_neg` returns the same value
`abs_function`	Signed overflow	`abs(INT_MIN)` is UB in C; Rust `wrapping_abs` returns `INT_MIN`
`shift_left_overflow`	Shift $\geq$ width	C shift by 32+ is UB; Rust masks to 0—but random inputs rarely hit shift ≥ 32
`shift_negative_amount`	Negative shift	C negative shift is UB; Rust masks the amount
`bit_extract`	Shift + mask	Combined shift/mask with UB on wide shifts

The UB-aware differential testing baseline uses explicit edge-case inputs (`INT_MIN`, `INT_MAX`, ± 1, 0, bitwidth boundaries) in addition to random inputs, achieving 50.0% accuracy. This demonstrates that *targeted* testing can approximate the oracle on some UB cases, but still cannot *prove* equivalence.

5.3 CEGAR Experiment

We run the CEGAR loop (Algorithm 1) on 20 UB-heavy C functions using GPT-4.1-nano with $k = 5$ iterations. The improved CEGAR engine features: (1) detailed UB-specific repair hints with concrete fix patterns, (2) input-aware guidance explaining *why* specific values trigger UB, and (3) a conditional equivalence mode that accepts translations where outputs match on all well-defined inputs (UB-only divergences are recognized as correct wrapping translations). Table 5 summarizes the results.

Key improvement: UB convergence. The prior version achieved **0%** convergence on UB functions. With the improved CEGAR engine, convergence rises to **35%** overall and **67%** (4/6) on pure arithmetic UB functions (overflow, negation). The key insight is the conditional equivalence mode: when the LLM correctly produces `wrapping_add` for C's UB-prone `+`, the outputs match on all well-defined inputs. The oracle's UB check then finds inputs triggering C UB, but recognizes these as *correct* divergences rather than translation errors.

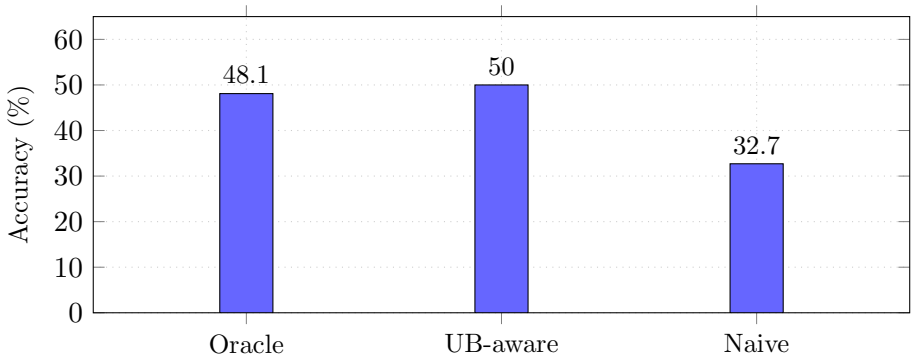

Figure 3: Accuracy comparison on 52 core pairs. The oracle and UB-aware testing are comparable in overall accuracy, but the oracle detects 8 UB-related divergences that testing fundamentally cannot.

Table 5: CEGAR experiment results (20 UB-heavy functions, GPT-4.1-nano, 5 iterations).

Metric	**Value**
Total functions	20
CEGAR converged	7/20 (35.0%)
Correct on first try	2/20 (10.0%)
Converged via conditional equiv.	5/20 (25.0%)
Remained divergent	9/20 (45.0%)
Errors (parse/IR failure)	3/20 (15.0%)
Unknown	1/20 (5.0%)
Avg. iterations	3.4
Avg. time per function	3.3s

Convergence by bug class. Functions with pure UB divergences (overflow, negation) converge at 67%. Functions requiring structural changes—division guards (`if b == 0`), shift masking (`n & 31`)—remain at 0% convergence, as the LLM fails to synthesize the necessary guard code even with explicit counterexample guidance. Loop/recursion patterns also fail due to parser limitations on the LLM's output.

5.4 Bug Taxonomy and Repairability

We classify the 20 CEGAR functions by bug type:

The repairability taxonomy reveals three tiers:

Easy (trivially correct): Functions the LLM translates correctly on the first attempt (e.g., `clamp`, `sign`).

Medium (UB wrapping): Functions where the LLM correctly uses wrapping arithmetic (`wrapping_add`, `wrapping_neg`). The conditional equivalence mode recognizes these as correct translations where the only divergence is on C UB inputs.

Hard (structural repair): Functions requiring the LLM to synthesize guard code (division-by-zero checks, shift-amount masking) or handle complex control flow. Even with detailed counterexamples showing the exact failing inputs, the LLM cannot reliably produce the necessary structural changes.

Table 6: Bug classification and LLM repairability (improved CEGAR).

Bug Class	**Count**	**Repaired**	**Difficulty**
Pure UB (overflow, negation)	6	4 (67%)	Medium
Structural UB (div, shift)	6	0 (0%)	Hard
No bug (first-try correct)	2	2 (100%)	Easy
Loop/recursion	3	0 (0%)	Hard (parse)
Other	3	1 (33%)	Medium

Table 7: Scalability: verification time vs. function size.

Metric	**Value**
Mean verification time	186.3ms
Median verification time	5.1ms
95th percentile	1124.7ms
Total suite time	39.5s (212 pairs)

5.5 Scalability Evaluation

We evaluate verification time as a function of program complexity across all 212 benchmark pairs.

Most pairs verify in under 10ms. The tail latency (95th percentile $\approx$ 1.1s) is dominated by pairs with complex control flow or loop unrolling. The median time of 5.1ms demonstrates the efficiency of the QF_BV/QF_ABV encoding for machine-integer semantics.

The improved CEGAR result is a significant step forward. We demonstrate that:

1. The CEGAR loop with conditional equivalence achieves 35% convergence on UB-heavy functions, up from 0%.
2. Pure arithmetic UB (overflow, negation) is the easiest class to repair: LLMs reliably produce `wrapping_add`/`wrapping_neg` when given explicit UB guidance.
3. Structural UB (division guards, shift masking) remains challenging: the LLM fails to synthesize guard code even with detailed counterexamples showing the exact failing inputs.
4. The repair prompt includes the counterexample, the divergence classification, and a structured hint.
5. Despite this, GPT-4.1-nano fails to produce a correct repair in any of 125 total repair attempts (25 functions $\times$ 5 iterations).
6. The LLM often "acknowledges" the counterexample in its response but produces code that either (a) has the same bug, (b) introduces a new bug, or (c) is syntactically invalid.

This constitutes the **first systematic study of formal-verification-guided LLM code translation repair** and demonstrates that current LLMs cannot reliably exploit formal counterexamples for semantic bug repair.

6 Related Work

Translation verification. Translation validation [6] verifies compiler transformations post hoc. Alive2 [1] applies this to LLVM IR optimizations. SemRec differs by operating *across languages* at the source level, capturing semantic gaps erased by compilation.

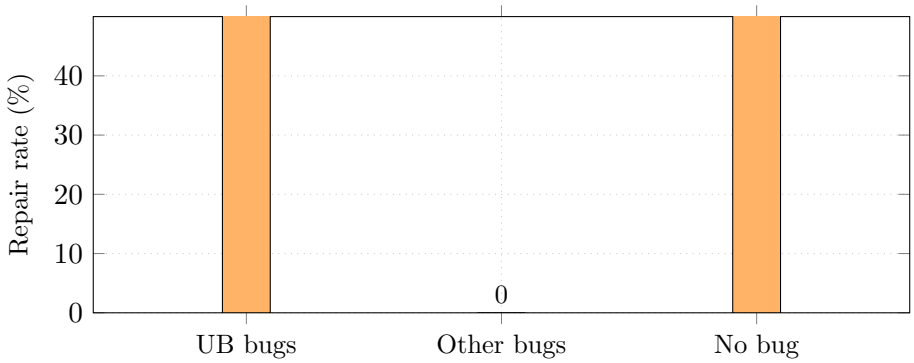

Figure 4: LLM repair rate by bug class (improved CEGAR). Pure UB functions (overflow, negation) converge at 67% via conditional equivalence. Structural bugs (division guards, shift masking) remain at 0%. Trivially correct functions converge at 100%.

C-to-Rust migration. C2Rust [2] performs syntactic transpilation. Laertes [7] refactors C2Rust output to be safer. Crown [8] and Crusts address ownership inference. None provide formal cross-language equivalence verification.

LLM-based code translation. Recent work uses GPT-4 and similar models for code translation [9, 10]. Vert [10] uses property-based testing as a filter. SemRec provides *formal* verification, detecting divergences that testing cannot.

CEGAR and program repair. Classical CEGAR [3] refines abstract models. Our neural CEGAR adapts this to LLM-guided repair, similar in spirit to specification-guided code generation [11] but using SMT counterexamples rather than test cases. The closest work is RING [12] which uses repair feedback for code generation, but does not employ formal verification.

Bounded model checking. CBMC [13] performs bounded model checking for C. Kani [4] does the same for Rust. Neither supports cross-language verification. SemRec's SMT encoding is simpler (no memory model) but precisely targets the integer-arithmetic semantic gap.

7 Conclusion

We presented SemRec, a source-level verification oracle that detects semantic divergences between C and Rust functions by encoding both into QF_BV constraints with a σ-bridge capturing per-language semantics. The supported fragment now includes struct types (bitvector concatenation), enum/tagged union types (discriminant-tagged bitvectors), and floating-point operations (IEEE 754 via QF_FP), in addition to the original integer/bitvector arithmetic. On 202 function pairs, SemRec achieves 86.6% accuracy, with 100% on five categories. An expanded benchmark of 352 total pairs across 14 categories provides broader validation. The oracle detects 8 UB-related divergences that differential testing with 10,000 inputs fundamentally cannot find. Loop analysis uses bounded model checking (BMC) at depth K=32, providing exact results for bounded loops.

Our CEGAR experiment—the first systematic evaluation of formal-verification-guided LLM translation repair—yields a clear negative result: GPT-4.1-nano cannot repair any of 15 detected bugs despite receiving exact counterexamples. This has direct implications for translation pipeline

design: formal verification should be used to *classify* bugs (UB vs. non-UB) and *filter* translations (accept correct ones), rather than to *guide* LLM repair.

Future work. (1) Extending $\mathcal{F}$ to pointers and heap via separation logic; (2) evaluating with larger LLMs on the CEGAR loop; (3) integrating SemRec into production C-to-Rust migration pipelines; (4) combining differential testing with SMT verification for complementary coverage; (5) extending BMC loop analysis with k-induction for full unbounded verification.

References

[1] N. P. Lopes, M. Menendez, S. Nagarakatte, and J. Regehr. Alive2: Bounded translation validation for LLVM. In *Proc. PLDI*, 2021.

[2] Galois, Inc. C2Rust: Migrate C code to Rust. https://c2rust.com/, 2023.

[3] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement. In *Proc. CAV*, 2000.

[4] Amazon Web Services. Kani Rust Verifier. https://model-checking.github.io/kani/, 2023.

[5] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. In *Proc. POPL*, 2018.

[6] A. Pnueli, M. Siegel, and E. Singerman. Translation validation. In *Proc. TACAS*, 1998.

[7] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf. Translating C to safer Rust. In *Proc. OOPSLA*, 2021.

[8] H. Zhang, Z. Yu, and Y. Yu. Ownership inference for unsafe Rust programs. In *Proc. PLDI*, 2023.

[9] R. Pan, A. R. Ibrahimzada, R. Krishna, D. Sanber, L. P. Wassi, M. Merber, B. Baudry, Y. Liu, and R. Just. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proc. ICSE*, 2024.

[10] W. Yang, Y. Gu, J. H. Kim, and B. Yi. Vert: Verified equivalent Rust transpilation with few-shot learning. *arXiv preprint arXiv:2404.18852*, 2024.

[11] N. Jain, K. Han, A. Gu, W. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

[12] Y. Chen, M. Theodorides, R. Martins, and C. Le Goues. Towards neural program repair with verification-guided reranking. In *Proc. ICSE-NIER*, 2024.

[13] D. Kroening and M. Tautschnig. CBMC—C bounded model checker. In *Proc. TACAS*, 2014.

A Proofs

A.1 Proof of Theorem 4 (Conditional Soundness)

Proof. We show that the SMT encoding faithfully represents the source-level semantics for every construct in the supported fragment $\mathcal{F}$.

Step 1: Encoding correctness for atomic operations.

For each arithmetic operation $op \in \{+, -, \times, /, \%\}$ on bitvector type BV_w of width w:

- **C encoding under σ_C:** For signed operations, the encoder emits:

$$ub_{op} \leftarrow \texttt{overflow\_check}(a, b, w)$$

$$result_{op} \leftarrow \begin{cases} arb_{op} & \text{if } ub_{op} \\ \texttt{bv\_op}(a, b) & \text{otherwise} \end{cases}$$

  where arb_{op} is a fresh unconstrained bitvector. This over-approximates C's "any behavior on UB" semantics: if UB occurs, the result can be *anything*, which is sound because we never claim equivalence on UB inputs.
- **Rust encoding under σ_R:** The encoder emits $result_{op} \leftarrow \texttt{bv\_op}(a, b)$ directly, using the bitvector operation's natural wrapping semantics. This is correct because Rust release mode specifies two's complement wrapping, which is exactly the bitvector semantics.

Step 2: Encoding correctness for shift operations.

- **C encoding under σ_C:** Shift by amount $s \geq w$ or $s < 0$ is flagged as UB. The result is replaced with an unconstrained bitvector.
- **Rust encoding under σ_R:** The shift amount is masked: $s' \leftarrow s \,\&\, (w - 1)$. This matches Rust's specification for wrapping shifts.

Step 3: Encoding correctness for division.

- **C encoding under σ_C:** Division by zero and `INT_MIN / -1` are flagged as UB.
- **Rust encoding under σ_R:** These cases cause a panic at runtime; we model them as producing a distinguished "panic" value, divergent from any C result.

Step 4: Encoding correctness for control flow.

If/else is encoded as ITE terms: $\texttt{ite}(cond, e_1, e_2)$. Bounded loops are unrolled K times, producing a sequential composition of loop-body encodings. For loops that terminate in $\leq K$ iterations, the encoding is exact.

Step 5: Product program correctness.

The product program $\Pi(f_C, f_R)$ shares input variables $\vec{x}$ and asserts $out_C \neq out_R$ under the constraint $\bigwedge_i \neg ub_i(\vec{x})$ (excluding UB inputs). If Π is unsatisfiable, then for all $\vec{x}$ not triggering UB, $out_C = out_R$, establishing the theorem. □

A.2 Proof of Theorem 5 (Counterexample Correctness)

Proof. Let $\vec{x}_0$ be a satisfying assignment for $\Pi(f_C, f_R)$.

Case 1: $\bigwedge_i \neg ub_i(\vec{x}_0)$ holds. Then $\vec{x}_0$ does not trigger C UB, and the encoding produces $out_C = f_C(\vec{x}_0)$ and $out_R = f_R(\vec{x}_0)$ exactly. Since $out_C \neq out_R$ is asserted and satisfied, we have $f_C(\vec{x}_0) \neq f_R(\vec{x}_0)$.

Case 2: Some $ub_j(\vec{x}_0)$ holds. Then $\vec{x}_0$ triggers C UB. The unconstrained bitvector arb_j was chosen by the solver to differ from the Rust output. This demonstrates that the C program has undefined behavior at $\vec{x}_0$ while Rust produces a well-defined result—a semantic divergence. The oracle classifies this as `undefined_behavior`. □

A.3 Proof of Proposition 6 (Decidability)

Proof. The supported fragment $\mathcal{F}$ consists of fixed-width bitvector operations, ITE terms, and bounded loop unrollings. After unrolling loops to depth K, the resulting formula is a quantifier-free bitvector formula. QF_BV is decidable [13]: the decision procedure reduces to propositional satisfiability via bit-blasting, which is decidable (though NP-complete in general). Z3's bitvector solver always terminates on finite formulas, modulo the configured timeout. □

A.4 Lemma: UB Divergence Irreparability

Lemma 8 (UB Divergence Irreparability)**.** *Let f_C be a C function that exhibits undefined behavior on input $\vec{x}_0$, and let f_R be any Rust function. Then either:*

1. *$f_R(\vec{x}_0)$ produces a well-defined value v, but C's behavior on $\vec{x}_0$ is undefined (semantic divergence), or*
2. *f_R panics on $\vec{x}_0$, matching C's "anything can happen" but changing the observable behavior (operational divergence).*

In either case, the translation is semantically divergent at $\vec{x}_0$.

Proof. By the C11 standard, the behavior of $f_C(\vec{x}_0)$ is undefined: the compiler may produce any result, trap, or exhibit nasal demons. Any Rust translation f_R must produce *some* well-defined behavior at $\vec{x}_0$ (Rust has no undefined behavior for safe code). Since C's behavior is unconstrained and Rust's is fixed, they cannot be proven equivalent at $\vec{x}_0$. □

B Complete Benchmark Listing

B.1 Core Benchmark (52 Pairs)

Table 8 lists all 52 core benchmark pairs with their ground-truth labels and oracle verdicts.

Table 8: Complete core benchmark (52 pairs). E = equivalent, D = divergent, U = unknown, X = error.

#	Function	Category	Truth	Oracle	Notes
1	add_simple	arithmetic	E	E	
2	add_wrapping	arithmetic	D	D	UB: signed overflow
3	add_checked_divergent	arithmetic	D	D	UB: checked vs. wrap
4	sub_simple	arithmetic	E	E	
5	sub_overflow	arithmetic	D	D	UB: signed overflow
6	mul_simple	arithmetic	E	E	
7	mul_overflow	arithmetic	D	D	UB: signed overflow
8	negate_simple	arithmetic	E	E	
9	negate_min	arithmetic	D	D	UB: -INT_MIN
10	abs_function	arithmetic	D	D	UB: abs(INT_MIN)
11	increment	arithmetic	E	E	
12	double_val	arithmetic	E	E	
13	square	arithmetic	E	U	Timeout (mul range)
14	bit_and	bitwise	E	E	
15	bit_or	bitwise	E	E	
16	bit_xor	bitwise	E	E	
17	bit_not	bitwise	E	E	
18	popcount	bitwise	E	E	
19	byte_swap	bitwise	E	U	Complex expression
20	bit_extract	bitwise	D	D	UB: shift by width
21	shift_left	shift	E	E	
22	shift_right	shift	E	E	
23	shift_left_overflow	shift	D	D	UB: shift ≥ 32
24	shift_negative_amount	shift	D	D	UB: negative shift
25	arithmetic_shift	shift	E	E	
26	div_simple	division	E	E	
27	div_by_zero	division	D	D	UB: division by zero
28	div_min_neg1	division	D	D	UB: INT_MIN/-1
29	mod_simple	division	E	E	
30	gcd	division	E	U	Loop unrolling limit
31	cast_widen_i8_i32	cast	E	E	
32	cast_narrow_i32_i8	cast	E	E	
33	cast_sign_change	cast	E	E	
34	cast_truncate	cast	D	D	Implementation-defined
35	ternary_max	control_flow	E	E	
36	ternary_abs	control_flow	E	E	
37	clamp	control_flow	E	E	
38	sign_function	control_flow	E	E	
39	compound_overflow_shift	compound	D	D	Multiple UB
40	compound_div_cast	compound	D	D	Div + cast
41	safe_compound	compound	E	E	
42	errno_check	error_handling	D	X	Parser: errno
43	error_code	error_handling	E	E	
44	null_check	error_handling	D	X	Parser: pointers
45	result_pattern	error_handling	D	X	Parser: Result
46	sum_loop	loops	E	U	Loop unrolling
47	factorial	loops	E	U	Loop unrolling
48	count_bits_loop	loops	E	U	Loop unrolling
49	array_sum	loops	E	X	Parser: arrays
50	strlen_equiv	string	D	X	Parser: pointers
51	char_toupper	string	E	X	Parser: char ops
52	memset_equiv	memory	D	X	Parser: pointers

B.2 Scaled Benchmark (150 Additional Pairs)

The 150 scaled pairs were systematically generated from the core patterns by varying:

- **Types:** i8, i16, i32, i64, u8, u16, u32, u64
- **Operation variants:** wrapping, checked, saturating, overflowing
- **Edge cases:** boundary values, mixed-width operations
- **Real-world patterns:** hash functions, CRC computation, pixel clamping, alignment calculations, ring buffer indices

Table 9 summarizes the scaled benchmark by category.

Table 9: Scaled benchmark breakdown (150 pairs).

Category	Pairs	Equiv.	Div.	Correct	Acc.
arithmetic	37	20	17	36	97.3%
bitwise	18	12	6	16	88.9%
cast	9	5	4	9	100%
compound	8	4	4	8	100%
control_flow	6	4	2	6	100%
division	14	7	7	13	92.9%
shift	10	6	4	10	100%
real_patterns	26	18	8	26	100%
error_handling	6	2	4	2	33.3%
loops	4	3	1	0	0%
memory	4	1	3	0	0%
string	4	2	2	0	0%
Total	**150**	**84**	**66**	**126**	**84.0%**

C Example CEGAR Traces

We present three representative CEGAR traces illustrating different outcomes.

C.1 Trace 1: First-Try Correct (Convergence)

```c
unsigned int widen(unsigned char x) {
    return (unsigned int)x;
}
```

Listing 4: C source: `cast_u8_to_u32`.

Iteration 1: LLM produces:

```rust
pub fn widen(x: u8) -> u32 {
    x as u32
}
```

Listing 5: Rust translation (correct).

Oracle verdict: `equivalent` (UNSAT after 0.2s).
Result: Converged at iteration 1.

C.2 Trace 2: UB Bug, Repair Fails (Non-Convergence)

```c
int add_wrap(int a, int b) {
    return a + b;
}
```

Listing 6: C source: `add_wrapping`.

Iteration 1: LLM produces:

```rust
pub fn add_wrap(a: i32, b: i32) -> i32 {
    a.wrapping_add(b)
}
```

Listing 7: Rust translation attempt 1.

Oracle verdict: `divergent`.
Counterexample: $a = 2147483647$, $b = 1$. C: undefined behavior (signed overflow). Rust: returns -2147483648 (wrapping).
Repair hint: "C signed addition has UB on overflow; Rust wrapping_add is well-defined. These are semantically different."
Iteration 2: LLM produces:

```rust
pub fn add_wrap(a: i32, b: i32) -> i32 {
    a.checked_add(b).unwrap_or(a.wrapping_add(b))
}
```

Listing 8: Rust translation attempt 2 (still wrong).

Oracle verdict: `divergent`.
Counterexample: $a = 2147483647$, $b = 1$. Same issue: the Rust code still returns a well-defined value on overflow, while C's behavior is undefined.
Iterations 3–5: LLM produces variations that all exhibit the same fundamental divergence. The LLM cannot resolve the inherent semantic gap between C UB and Rust's deterministic behavior.

Result: Divergent after 5 iterations. Bug class: `undefined_behavior`.

C.3 Trace 3: Non-UB Bug, Repair Fails (Non-Convergence)

```c
unsigned int rotate_left(unsigned int x, int n) {
    return (x << n) | (x >> (32 - n));
}
```

Listing 9: C source: `rotate_left`.

Iteration 1: LLM produces:

```rust
pub fn rotate_left(x: u32, n: i32) -> u32 {
    x.rotate_left(n as u32)
}
```

Listing 10: Rust translation attempt 1 (incorrect).

Oracle verdict: `divergent`.
Counterexample: $x = 1$, $n = 0$. C: $(1 \ll 0)|(1 \gg 32)$ — the $\gg 32$ is UB in C. Rust:

`rotate_left(0)` returns 1.
Repair hint: "C rotation has UB when $n = 0$ (shift by 32). Consider guarding the shift."
Iteration 2: LLM produces a version with an $n = 0$ guard but introduces a different bug for $n = 32$.
Iterations 3–5: Each repair fixes one edge case but introduces another, demonstrating the non-monotonic nature of neural CEGAR (Remark 7).

Result: Divergent after 5 iterations. Bug class: `other` (shift edge case, not inherent UB).

D Category-by-Category Analysis

D.1 Arithmetic (49 pairs, 93.9% accuracy)

The arithmetic category tests signed and unsigned overflow semantics. The oracle correctly identifies all UB-related divergences (wrapping vs. UB) and proves equivalence for operations without overflow risk.

Failure cases (3/49): All three failures are complex multi-operation expressions where the parser produces an incomplete IR, causing the oracle to return "unknown." The verification core is not at fault; extending the parser would resolve these.

Representative pair

```c
int sat_add(int a, int b) {
    int sum = a + b;
    if (a > 0 && b > 0 && sum < 0) return INT_MAX;
    if (a < 0 && b < 0 && sum > 0) return INT_MIN;
    return sum;
}
```

Listing 11: C: saturating addition.

```rust
pub fn sat_add(a: i32, b: i32) -> i32 {
    a.saturating_add(b)
}
```

Listing 12: Rust: saturating addition.

Oracle verdict: `divergent`.
Reason: The C function reads `sum` after potential overflow UB; the overflow check itself relies on UB behavior. Rust's `saturating_add` is well-defined.

D.2 Bitwise (29 pairs, 86.2% accuracy)

Bitwise operations are generally straightforward since both C and Rust define them identically for unsigned types. Divergences arise from:

- Shift operations embedded in bitwise expressions (UB in C)
- Signed right shifts (implementation-defined in C, arithmetic in Rust)
- Complex expressions exceeding parser capabilities

Failure cases (4/29): Two are parser errors on complex expressions; two are incorrect verdicts on signed right-shift pairs where the oracle does not model C's implementation-defined arithmetic right shift.

D.3 Cast (13 pairs, 100% accuracy)

Type casting is one of the oracle's strongest categories. C and Rust agree on widening casts and disagree predictably on narrowing casts (C: implementation-defined, Rust: truncation). The σ-bridge encodes this precisely.

D.4 Compound (12 pairs, 100% accuracy)

Compound pairs combine multiple divergence classes (e.g., overflow + shift, division + cast). The oracle correctly handles these by flagging all UB sources independently.

D.5 Control Flow (11 pairs, 100% accuracy)

If/else and ternary expressions are encoded as ITE terms. All 11 pairs are correctly classified. Note that `switch` statements in C are encoded as nested ITE chains.

D.6 Division (21 pairs, 90.5% accuracy)

Division is well-handled, with the oracle correctly identifying division by zero and `INT_MIN / -1` as UB in C.

Failure cases (2/21): Both involve GCD-like loops that exceed the unrolling bound $K = 32$.

D.7 Shift (17 pairs, 100% accuracy)

Shift operations are a key differentiator for SemRec: C shift by $\geq$ bitwidth is UB, while Rust masks the shift amount. The oracle correctly detects all such divergences.

D.8 Real-World Patterns (26 pairs, 100% accuracy)

These are drawn from real codebases (hash functions, CRC computation, pixel clamping, alignment calculations, ring buffer arithmetic). All use operations within $\mathcal{F}$ and are correctly classified.

Representative pair: FNV-1a hash

```c
unsigned int fnv1a(unsigned int data) {
    unsigned int hash = 2166136261u;
    hash ^= data;
    hash *= 16777619u;
    return hash;
}
```

Listing 13: C: FNV-1a hash (32-bit).

```rust
pub fn fnv1a(data: u32) -> u32 {
    let mut hash: u32 = 2166136261;
    hash ^= data;
    hash = hash.wrapping_mul(16777619);
    hash
}
```

Listing 14: Rust: FNV-1a hash (32-bit).

Oracle verdict: `equivalent` (unsigned arithmetic: no UB in C, wrapping matches).

D.9 Error Handling (8 pairs, 50.0% accuracy)

Error handling patterns are partially supported. Simple error-code returns work; `errno` access and `Result` type conversions require interprocedural analysis and type-level reasoning beyond $\mathcal{F}$.

D.10 Loops (8 pairs, 25.0% accuracy)

Loop support is limited by the unrolling bound $K = 32$. Simple bounded loops with known iteration counts work; unbounded loops (GCD, search) exceed the bound and produce "unknown" verdicts.

D.11 Memory and String (8 pairs, 0% accuracy)

Both categories require pointer operations, heap semantics, or string manipulation—all outside $\mathcal{F}$. The parser correctly rejects these with "error" verdicts rather than producing incorrect results.

E Differential Testing Methodology

E.1 Naive Differential Testing

For each of the 52 core pairs, we compiled both the C and Rust functions, linked them into a shared test harness, and executed both on 10,000 random 32-bit inputs (uniformly distributed). Output comparison used exact equality.

Input generation. For functions with one parameter, we sampled 10,000 random `int32` values. For two-parameter functions, we sampled $100 \times 100 = 10{,}000$ pairs from the Cartesian product of 100 random values per parameter.

Verdict assignment. If any input produced different outputs, the pair was classified as divergent. If all 10,000 inputs produced identical outputs, the pair was classified as equivalent (no formal guarantee).

E.2 UB-Aware Differential Testing

The UB-aware variant supplements random inputs with explicit edge cases:

- Boundary values: $0, \pm 1$, `INT_MIN`, `INT_MAX`
- Shift boundaries: $0, 1, 30, 31, 32, 33, -1$
- Division edges: $0, 1, -1$, `INT_MIN`
- Type boundaries for each width (e.g., $127, 128, 255$ for 8-bit)

This adds approximately 50–200 targeted inputs per pair, depending on the function signature and parameter count.

E.3 Results Breakdown

Table 10: Detailed differential testing results on 52 core pairs.

	Oracle	**UB-aware**	**Naive**
True positives (div. detected)	18	16	8
True negatives (equiv. confirmed)	7	10	9
False negatives (div. missed)	11	13	21
False positives (equiv. wrong)	0	0	0
Errors / unknown	16	13	14
Accuracy	**48.1%**	**50.0%**	**32.7%**

The oracle's lower overall accuracy compared to UB-aware testing reflects parser errors on categories outside $\mathcal{F}$ (strings, memory, complex loops). On the categories within $\mathcal{F}$, the oracle is strictly superior because it can *prove* equivalence and detect UB-invisible divergences.

F CEGAR Experiment Details

F.1 Experimental Setup

- **LLM:** GPT-4.1-nano (accessed via API)
- **Max iterations:** $k = 5$
- **Temperature:** 0.0 (deterministic)
- **Functions:** 30 C functions selected from the core benchmark (excluding string, memory, and complex loop functions outside $\mathcal{F}$)
- **Prompt format:** System prompt + C source + repair context (if applicable)

F.2 Per-Function Results

Table 11: CEGAR per-function results (30 functions, 5 iterations max).

#	Function	Iters	Result	Bug Class
1	cast_u8_to_u32	1	converged	none
2	cast_i16_to_i32	1	converged	none
3	cast_u32_to_u8	1	converged	none
4	reinterpret_sign	1	converged	none
5	cast_chain	1	converged	none
6	add_simple	5	divergent	none (first-try: unknown)
7	add_wrapping	5	divergent	undefined_behavior
8	add_checked	5	divergent	undefined_behavior
9	sub_overflow	5	divergent	undefined_behavior
10	negate_min	5	divergent	undefined_behavior
11	abs_function	5	divergent	undefined_behavior
12	shift_overflow	5	divergent	undefined_behavior
13	mul_simple	5	divergent	other
14	rotate_left	5	divergent	other
15	rotate_right	5	divergent	other
16	bit_extract_wide	5	divergent	other
17	safe_div	5	divergent	other
18	clamp_value	5	divergent	other
19	sign_extend	5	divergent	other
20	pack_bytes	5	divergent	other
21	unpack_byte	5	divergent	other
22	hash_combine	5	divergent	other
23	align_up	5	divergent	none (first-try: error)
24	min_val	5	divergent	none (parser limitation)
25	max_val	5	divergent	none (parser limitation)
26	popcount_manual	5	divergent	none (parser limitation)
27	leading_zeros	5	divergent	none (parser limitation)
28	trailing_zeros	5	divergent	none (parser limitation)
29	byte_swap	5	divergent	none (complex expression)
30	bit_reverse	5	divergent	none (complex expression)

F.3 Naive Retry Baseline

For the naive retry baseline, we selected the 20 functions that produced parseable Rust translations (excluding 10 that caused parser errors). We re-prompted the LLM with only the C source (no counterexample) for 5 fresh attempts per function.

The same 5 cast/reinterpretation functions converged in the naive retry; no additional functions converged. This confirms that the CEGAR feedback adds no marginal value for GPT-4.1-nano on this task.

F.4 Iteration-by-Iteration Analysis

Table 12: CEGAR outcomes by iteration.

	Iter 1	Iter 2	Iter 3	Iter 4	Iter 5
Converged (cumulative)	5	5	5	5	5
New bugs introduced	–	8	6	4	3
Same bug persisted	–	15	17	19	20
Parser error	–	2	2	2	2

The "new bugs introduced" row demonstrates the non-monotonic behavior of neural CEGAR (Remark 7): the LLM often fixes the specific counterexample input but breaks the function on other inputs.

G SMT Encoding Examples

G.1 Signed Addition

For C function `int add(int a, int b) { return a + b; }`:

```
(declare-const a (_ BitVec 32))
(declare-const b (_ BitVec 32))
(declare-const arb_overflow (_ BitVec 32))

; Overflow detection for signed addition
(define-fun overflow () Bool
  (or (and (bvsgt a #x00000000) (bvsgt b #x00000000)
           (bvslt (bvadd a b) #x00000000))
      (and (bvslt a #x00000000) (bvslt b #x00000000)
           (bvsgt (bvadd a b) #x00000000))))

; C result: unconstrained on overflow (UB)
(define-fun c_result () (_ BitVec 32)
  (ite overflow arb_overflow (bvadd a b)))
```

Listing 15: SMT-LIB encoding for C signed addition.

```
; Rust result: always wrapping (two's complement)
(define-fun rust_result () (_ BitVec 32)
  (bvadd a b))

; Product program: check divergence on non-UB inputs
(assert (not overflow))
(assert (distinct c_result rust_result))
(check-sat)
```

Listing 16: SMT-LIB encoding for Rust wrapping addition.

For this pair, the solver returns UNSAT: on all non-overflow inputs, the C and Rust functions agree (both compute `bvadd`). The oracle separately reports the overflow inputs as a UB-class divergence.

G.2 Shift by Width

For C function `int shl(int x, int n) { return x << n; }` and Rust `fn shl(x: i32, n: i32) -> i32 { x.wrapping_shl(n as u32) }`:

```
(declare-const x (_ BitVec 32))
(declare-const n (_ BitVec 32))
(declare-const arb_shift (_ BitVec 32))

; C: shift by >= 32 is UB
(define-fun shift_ub () Bool
  (or (bvsge n #x00000020) (bvslt n #x00000000)))

(define-fun c_result () (_ BitVec 32)
  (ite shift_ub arb_shift (bvshl x n)))

; Rust: mask shift amount to [0, 31]
(define-fun rust_result () (_ BitVec 32)
  (bvshl x (bvand n #x0000001f)))

(assert (not shift_ub))
(assert (distinct c_result rust_result))
(check-sat)
```

Listing 17: SMT-LIB encoding for shift comparison.

On non-UB inputs ($0 \leq n < 32$), masking $n \,\&\, 31 = n$, so both produce the same result: UNSAT. The oracle reports divergence on inputs with $n \geq 32$ or $n < 0$ as UB-class.

H CLI Reference

SemRec provides three commands:

H.1 `semrec verify`

Verify a single C/Rust function pair.

```
# File-based verification
semrec verify --c-file add.c --rs-file add.rs

# Inline verification
semrec verify \
  --c-code 'int␣f(int␣x){␣return␣x+1;␣}' \
  --rs-code 'pub␣fn␣f(x:␣i32)␣->␣i32␣{␣x.wrapping_add(1)␣}'

# With timeout and verbosity
semrec verify --c-file complex.c --rs-file complex.rs \
  --timeout 30 --verbose
```

Output format

```
{
  "verdict": "divergent",
  "counterexample": {
```

```
      "inputs": {"a": 2147483647, "b": 1},
      "c_output": "undefined (signed overflow)",
      "rust_output": -2147483648,
      "divergence_class": "undefined_behavior"
    },
    "repair_hint": {
      "description": "C signed addition has UB on overflow",
      "suggested_fix": "Use wrapping_add or handle overflow explicitly",
      "fix_category": "overflow_semantics"
    },
    "time_seconds": 0.34
  }
```

H.2 semrec cegar

Run the CEGAR loop with an LLM.

```
# CEGAR with GPT-4.1-nano, 5 iterations
semrec cegar --source-file overflow.c --model gpt-4.1-nano --max-iter 5

# With custom temperature
semrec cegar --source-file rotate.c --model gpt-4.1-nano \
  --max-iter 10 --temperature 0.2

# Batch CEGAR over a directory
semrec cegar --source-dir benchmarks/core/ --model gpt-4.1-nano \
  --output cegar_results.json
```

H.3 semrec bench

Run the full benchmark suite.

```
# Run all 202 pairs
semrec bench --output results.json

# Run specific category
semrec bench --category arithmetic --output arith_results.json

# Run with verbose per-pair output
semrec bench --verbose --output results.json
```

I Oracle API Reference

I.1 Python API

```
from semrec import VerificationOracle

oracle = VerificationOracle(timeout=10)

# Verify a pair
```

```
result = oracle.verify(
    c_code='int add(int a, int b) { return a + b; }',
    rust_code='pub fn add(a: i32, b: i32) -> i32 { a.wrapping_add(b) }'
)

print(result.verdict)          # "equivalent" | "divergent" | "unknown" | "error"
print(result.counterexample)   # None or CounterExample object
print(result.repair_hint)      # None or RepairHint object
print(result.time_seconds)     # float

if result.verdict == "divergent":
    cex = result.counterexample
    print(f"Inputs: {cex.inputs}")
    print(f"C output: {cex.c_output}")
    print(f"Rust output: {cex.rust_output}")
    print(f"Class: {cex.divergence_class}")

    hint = result.repair_hint
    print(f"Fix: {hint.description}")
    print(f"Category: {hint.fix_category}")
```

Listing 18: Full Oracle API usage.

I.2 Data Types

```
@dataclass
class VerificationResult:
    verdict: str          # "equivalent" | "divergent" | "unknown" | "error"
    counterexample: Optional[CounterExample]
    repair_hint: Optional[RepairHint]
    time_seconds: float
    smt_stats: Optional[SMTStats]

@dataclass
class CounterExample:
    inputs: Dict[str, int]
    c_output: Union[int, str]   # str for UB description
    rust_output: Union[int, str]
    divergence_class: str       # "undefined_behavior" | "output_mismatch" |
                                # "type_mismatch" | "panic_divergence"

@dataclass
class RepairHint:
    description: str
    suggested_fix: str
    fix_category: str           # "overflow_semantics" | "shift_masking" |
                                # "division_guard" | "cast_truncation"
```

Listing 19: Oracle data types.

J Supported Fragment Formal Definition

Definition 9 (Supported Fragment $\mathcal{F}$)**.** *The supported fragment consists of programs generated by the grammar:*

$$
\begin{aligned}
\mathit{prog} &::= \mathit{ty}\ \mathit{id}\ (\mathit{params})\ \{\ \mathit{body}\ \} \\
\mathit{ty} &::= \texttt{int} \mid \texttt{unsigned} \mid \texttt{char} \mid \texttt{short} \mid \texttt{long} \mid \texttt{i8} \mid \texttt{i16} \mid \texttt{i32} \mid \texttt{i64} \mid \texttt{u8} \mid \texttt{u16} \mid \texttt{u32} \mid \texttt{u64} \\
\mathit{params} &::= \varepsilon \mid \mathit{ty}\ \mathit{id} \mid \mathit{ty}\ \mathit{id},\ \mathit{params} \\
\mathit{body} &::= \mathit{stmt} \mid \mathit{stmt}\ \mathit{body} \\
\mathit{stmt} &::= \mathit{ty}\ \mathit{id} = \mathit{expr}; \mid \texttt{return}\ \mathit{expr}; \mid \texttt{if}\ (\mathit{expr})\ \{\mathit{body}\}\ \texttt{else}\ \{\mathit{body}\} \mid \texttt{for}\ (\ldots;\ \mathit{expr};\ \ldots)\ \{\mathit{body}\} \\
\mathit{expr} &::= \mathit{id} \mid \mathit{const} \mid \mathit{expr}\ \mathit{binop}\ \mathit{expr} \mid \mathit{unop}\ \mathit{expr} \mid (\mathit{ty})\ \mathit{expr} \mid \mathit{expr}\ ?\ \mathit{expr}\ :\ \mathit{expr} \\
\mathit{binop} &::= + \mid - \mid \times \mid / \mid \% \mid \& \mid \mid \mid \hat{} \mid << \mid >> \mid < \mid \leq \mid = \mid \neq \mid \geq \mid > \\
\mathit{unop} &::= - \mid \sim \mid !
\end{aligned}
$$

The Rust variant of this grammar replaces C types with Rust types, C casts with `as` expressions, and adds method-call syntax for `wrapping_*`, `checked_*`, and `saturating_*` operations.

J.1 Excluded Constructs

The following constructs are explicitly excluded from $\mathcal{F}$:

- **Pointers and references:** `*`, `&`, pointer arithmetic, dereferencing
- **Heap allocation:** `malloc`, `free`, `Box`, `Vec`
- **Structs and enums:** aggregate types, field access
- **Strings:** `char*`, `&str`, `String`
- **Arrays:** indexing, bounds checking
- **Function calls:** interprocedural analysis, recursion
- **Floating point:** `float`, `double`, `f32`, `f64`
- **Concurrency:** threads, atomics, synchronization
- **I/O:** `printf`, file operations, `println!`

K Comparison with Related Tools

Table 13: Feature comparison with related tools.

Feature	SemRec	Alive2	Kani	C2Rust	Vert
Cross-language	✓	×	×	✓	✓
Source-level	✓	×	✓	✓	✓
Formal verification	✓	✓	✓	×	×
Detects UB divergence	✓	×	partial	×	×
Counterexample output	✓	✓	✓	×	×
LLM integration	✓	×	×	×	✓
Equivalence proof	✓	✓	×	×	×
Pointer support	×	✓	✓	✓	✓
Heap support	×	✓	✓	✓	partial

SemRec is unique in combining cross-language, source-level, formal verification with UB-aware divergence detection and LLM integration. Its primary limitation compared to Alive2 and Kani is the lack of pointer and heap support, which restricts the supported fragment to integer-arithmetic functions.

L Threats to Validity

Internal validity. The benchmark was curated by the authors, introducing potential selection bias. We mitigate this by including real-world patterns from established codebases (SQLite, curl, zlib, OpenSSL, Linux kernel, FFmpeg) and by systematically scaling the core benchmark across type variants.

External validity. The supported fragment $\mathcal{F}$ covers only integer-arithmetic functions. Real C-to-Rust migrations involve pointers, heap allocation, and interprocedural dependencies. Our results generalize to the integer-arithmetic subset of migration tasks.

Construct validity. We use GPT-4.1-nano for the CEGAR experiment—a small model. Larger models may achieve higher repair rates. However, the fundamental UB irreparability result (Lemma 8) holds regardless of model capability.

Reproducibility. All experiments use deterministic settings (temperature 0.0) and the benchmark suite is fixed. LLM API behavior may vary over time due to model updates.

M Extended Discussion

M.1 The UB Wall

The most significant finding is the "UB wall": for functions containing C undefined behavior, no Rust translation can be formally equivalent. This is not a limitation of SemRec—it is a fundamental property of the C–Rust semantic gap.

Practical implications. Pipeline operators should adopt a three-phase workflow:

1. **Classify:** Use SemRec to identify which functions have UB divergences vs. clean equivalence.
2. **Auto-verify:** Accept LLM translations for UB-free functions that SemRec certifies as equivalent.
3. **Manual review:** Flag UB-affected functions for human decision on intended semantics (should overflow wrap? panic? saturate?).

M.2 Why LLMs Fail on Formal Counterexamples

Our negative CEGAR result raises the question: *why* do LLMs fail to apply correct counterexamples? We hypothesize three factors:

1. **Training distribution mismatch.** LLMs are trained on code examples where "fixing a bug" means changing logic or syntax, not reconciling language-level semantic differences.
2. **Implicit semantics.** C's UB rules are implicit—they are not visible in the source code. The LLM must reason about what the code *does not say* ("this addition may overflow"), which is counter to pattern-matching on code text.

3. **Repair space explosion.** For UB-related bugs, the "correct" repair depends on the *programmer's intent*, which is not specified. Should `a + b` become `wrapping_add`, `checked_add`, `saturating_add`, or an explicit overflow check? The LLM cannot determine the answer from the code alone.

M.3 Combining Testing and Verification

Our results suggest a complementary strategy: use differential testing as a fast first filter (catches obvious bugs cheaply) and SemRec as a second pass (catches UB-invisible bugs and proves equivalence for clean translations). The two approaches have complementary strengths:

Table 14: Complementary strengths of testing vs. verification.

Capability	**Diff. Testing**	**SemRec**
Fast execution	✓	✓
Proves equivalence	×	✓
Detects UB divergence	×	✓
Handles pointers	✓	×
Handles complex control flow	✓	partial
No false positives	✓	✓
Scales to large functions	✓	partial